

BME280 Environmental Sensor

Project Reference — Temperature, Humidity & Pressure

1. Overview

The BME280 is a combined digital humidity, pressure, and temperature sensor from Bosch Sensortec. It succeeds earlier sensors like the BMP180/BMP085 and is register-compatible with the BMP280. Your GY-BME280 module is a 3.3V breakout board carrying the BME280 IC along with support components. It is widely used for weather stations, indoor air monitoring, IoT projects, and altimeter applications.

Key capabilities:

- Temperature measurement with integrated ADC
- Barometric (absolute) pressure measurement with very low noise
- Relative humidity measurement with fast response time
- Estimated altitude from pressure (requires sea-level reference pressure)
- Two communication interfaces: I²C (up to 3.4 MHz) and SPI (3- or 4-wire, up to 10 MHz)
- Three operating modes: sleep, forced, normal
- Built-in IIR filter to suppress short-term disturbances

2. Electrical & Environmental Specifications

Parameter	Value
Supply voltage (VDD)	1.71 V to 3.6 V
Interface voltage (VDDIO)	1.2 V to 3.6 V
Module operating voltage (GY-BME280)	3.3 V (has onboard regulator)
Current — normal mode, 1 Hz	~3.6 μ A
Current — sleep mode	~0.1 μ A
Operating temperature range	–40 °C to +85 °C
Humidity range	0 % to 100 % RH (non-condensing)
Humidity accuracy	± 3 % RH
Pressure range	300 hPa to 1100 hPa
Pressure absolute accuracy	± 1 hPa
Temperature accuracy	± 1.0 °C
Pressure resolution (filtered)	20-bit (≈ 0.18 Pa)
Package	8-pin LGA, 2.5 × 2.5 × 0.93 mm
I ² C addresses	0x76 (default) or 0x77

Warning: The BME280 IC itself is a 3.3 V part. The GY-BME280 breakout usually includes a 3.3 V LDO and level shifter, so many versions accept 3.3–5 V on VIN. Verify with your specific board — applying 5 V to a board without a regulator will permanently damage the chip.

3. Module Pinout (GY-BME280)

The typical GY-BME280 breakout exposes 6 pins. Only 4 are needed for I²C, which is the common choice for most projects.

Pin	I ² C function	SPI function	Connect to
VIN / VCC	Power supply	Power supply	3.3 V (or 5 V w/ regulator)
GND	Ground	Ground	GND
SCL / SCK	I ² C clock	SPI clock	SCL (MCU)
SDA / SDI / MOSI	I ² C data	SPI MOSI (data in)	SDA (MCU)
CSB	Tie to VCC for I ² C	SPI chip select (CS)	VCC (I ² C) or GPIO (SPI)
SDO / MISO	Sets I ² C address	SPI MISO (data out)	GND = 0x76, VCC = 0x77

I²C address tip: the SDO pin determines the address. Pulling it to GND gives 0x76, pulling it to VCC gives 0x77. If the sensor is not detected at 0x76, try 0x77 — clone boards vary. An I²C scanner sketch is the fastest way to confirm.

4. Wiring Diagrams

4.1 I²C Wiring (4 wires, recommended)

BME280 pin	Arduino UNO	ESP32 (default)	Raspberry Pi (40-pin)
VIN	3.3V	3V3	Pin 1 (3.3V)
GND	GND	GND	Pin 6 (GND)
SCL	A5 (SCL)	GPIO 22	Pin 5 (GPIO 3 / SCL1)
SDA	A4 (SDA)	GPIO 21	Pin 3 (GPIO 2 / SDA1)

4.2 SPI Wiring (4-wire)

BME280 pin	Signal	Arduino UNO (hw SPI)
VIN	Power	3.3V
GND	Ground	GND
SCK (SCL)	SPI clock	D13 (SCK)
SDI (SDA)	MOSI	D11 (MOSI)
SDO	MISO	D12 (MISO)
CSB	Chip select	D10 (any GPIO)

5. Operating Modes

The BME280 supports three modes, selected via the **mode[1:0]** bits of the CTRL_MEAS register (0xF4):

- **Sleep mode** — default after power-up. No measurements taken; lowest current draw ($\sim 0.1 \mu\text{A}$).
- **Forced mode** — performs a single measurement, stores the result in the data registers, then returns to sleep. Best for low-duty-cycle battery applications.
- **Normal mode** — cycles continuously between measurement and a configurable standby (inactive) period. Best for real-time logging.

An internal IIR filter can be enabled to smooth out short-term disturbances (slamming doors, wind gusts) without extra host processing. Oversampling settings for temperature, pressure, and humidity are independently configurable.

6. Key Registers

The BME280 is controlled through a small set of registers. Reading and writing these directly is only required when not using a library — most Arduino, Python, and MicroPython libraries handle register access automatically.

Address	Register	Purpose
0xD0	id	Chip ID (reads 0x60 for BME280)
0xE0	reset	Write 0xB6 to perform a soft reset
0xF2	ctrl_hum	Humidity oversampling (write before ctrl_meas)
0xF3	status	Measuring & im_update flags
0xF4	ctrl_meas	Temperature & pressure oversampling, mode
0xF5	config	Standby time, IIR filter, SPI 3-wire enable
0xF7–0xF9	press	Pressure ADC data (20-bit)
0xFA–0xFC	temp	Temperature ADC data (20-bit)
0xFD–0xFE	hum	Humidity ADC data (16-bit)
0x88–0xA1, 0xE1–0xE7	calib	Factory calibration coefficients

Important: Raw ADC values must be compensated using the per-chip calibration coefficients. Bosch publishes reference compensation formulas in the datasheet — most libraries (Adafruit_BME280, Bosch BME280 driver) do this automatically.

7. Example Code

7.1 Arduino (Adafruit_BME280 library, I²C)

Install the **Adafruit BME280 Library** and **Adafruit Unified Sensor** via the Arduino IDE Library Manager.

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

#define SEALEVELPRESSURE_HPA (1013.25)

Adafruit_BME280 bme; // I2C

void setup() {
  Serial.begin(9600);
  while (!Serial);

  Serial.println("BME280 test");

  // Default I2C address is 0x76; try 0x77 if not found
  if (!bme.begin(0x76)) {
    Serial.println("No BME280 found - check wiring / address!");
    while (1) delay(10);
  }
}

void loop() {
  Serial.print("Temperature = ");
  Serial.print(bme.readTemperature());
  Serial.println(" *C");

  Serial.print("Pressure = ");
  Serial.print(bme.readPressure() / 100.0F);
  Serial.println(" hPa");

  Serial.print("Approx. altitude = ");
  Serial.print(bme.readAltitude(SEALEVELPRESSURE_HPA));
  Serial.println(" m");

  Serial.print("Humidity = ");
  Serial.print(bme.readHumidity());
  Serial.println(" %");

  Serial.println();
  delay(2000);
}
```

7.2 MicroPython (I²C)

```
from machine import I2C, Pin
from time import sleep
import bme280 # install with: upip.install('micropython-bme280')

i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=100000)
sensor = bme280.BME280(i2c=i2c, address=0x76)

while True:
    temp, pressure, humidity = sensor.read_compensated_data()
    print("T: {:.2f} C".format(temp / 100))
    print("P: {:.2f} hPa".format(pressure / 25600))
    print("H: {:.2f} %".format(humidity / 1024))
    sleep(2)
```

7.3 Raspberry Pi (Python, Linux)

Enable I²C with **raspi-config**, then install: `pip install RPi.bme280 smbus2`

```
import smbus2
import bme280

port = 1          # I2C bus 1 on Pi 3/4/5
address = 0x76    # or 0x77

bus = smbus2.SMBus(port)
calibration = bme280.load_calibration_params(bus, address)

while True:
    data = bme280.sample(bus, address, calibration)
    print(f"Temp:      {data.temperature:.2f} C")
    print(f"Pressure: {data.pressure:.2f} hPa")
    print(f"Humidity: {data.humidity:.2f} %")
    print()
```


8. Troubleshooting

Sensor not detected at 0x76: Try 0x77. Different modules ship with different default addresses. Run an I²C scanner sketch to confirm.

Readings are wildly wrong: Check that VIN is within tolerance and the chip isn't being heated by a nearby regulator. Re-seat SDA/SCL wires.

Temperature reads too high: Onboard regulators and nearby circuitry can heat the sensor. Mount the module away from heat sources or apply a software offset.

Altitude reading drifts: Altitude is calculated from pressure vs. a reference sea-level pressure. Update SEALEVELPRESSURE_HPA with your local METAR value.

Communication unstable over long wires: Use shorter leads, lower the I²C speed (e.g., 100 kHz), and ensure pullups are present (the module usually has them).

Device is found but returns all 0xFF: Often caused by 5 V applied to a 3.3V-only module. Check your board's voltage regulator. The IC may be damaged.

9. References & Further Reading

- **Bosch Sensortec — BME280 official datasheet (PDF)**

<https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>

- **Bosch Sensortec — BME280 product page**

<https://www.bosch-sensortec.com/products/environmental-sensors/humidity-sensors-bme280/>

- **Adafruit BME280 Library (GitHub)**

https://github.com/adafruit/Adafruit_BME280_Library

- **Bosch BME280 C driver (GitHub)**

https://github.com/boschsensortec/BME280_SensorAPI

- **SparkFun BME280 Hookup Guide**

<https://learn.sparkfun.com/tutorials/sparkfun-bme280-breakout-hookup-guide>

- **Random Nerd Tutorials — BME280 with Arduino**

<https://randomnerdtutorials.com/bme280-sensor-arduino-pressure-temperature-humidity/>

Reference compiled from the official Bosch Sensortec BME280 datasheet and community documentation. Always consult the current official datasheet for authoritative specifications.